VOLTARAX

Code verification and running guide

2 November 2026 IST | Windows PowerShell | Branch: phase-11-gaps

Phase 11 required local work and gates passed. Three earlier packaging defects were fixed. Connected installation still requires setup and field verification.

Reviewed source commit: `7cf096cde6ccaadea1a0c69acac1ad32a905d5bb`. The document/evidence commit follows it; prior phase tags remain historical checkpoints.

Error reproduced	Correction and verification
AI image copied an empty configs folder absent from Git	Removed unused COPY; tracked build-input check passes
AI image omitted evaluation.py; orchestrator raised ModuleNotFoundError	Included module; staged runtime and four batch entry points import offline
Backend image omitted scripts/seed.js although npm advertises first-admin setup	Included guarded script; packaged entry-point check passes

The old root README also treated first-admin setup as a routine start command and described outdated hosting assumptions. It now separates preview, configured services and one-time setup. The packaging check is included in CI; hosted CI has not run.

Fresh local verification	Result
Backend / AI cases	88 / 28 passed
Browser cases	10 public/UI + 4 actual API passed
Firebase rules / native helper cases	1 / 4 passed
Lint, source/generated types, production build	Passed
Firmware	Placeholder compile passed; no flash
Packaging, syntax, CI/Compose parsing, pattern scan	Passed within recorded scope
Production audit	Frontend 0; backend 8 moderate; no high/critical

Phase 11 adds reviewed EN/HI source-cited chat (provider off by default), original bounded flow media, savings-first grouped values, shared data states and expiry-aware IST headers. Axe measurements: 18 scans, 0 violations; incomplete/manual items remain recorded. Solid metric contrast minimum: 10.137:1 over 12 pairs. Optional nonce CSP: Skipped; inline allowances remain.

Evidence: docs/test-results/phase11-gate-summary.json and phase11-*.json/log. The preceding reverify-summary.json records the pre-Phase-11 second review. Tests used the existing locked installations, not a fresh reinstall. Automated checks and targeted startup/packaging review cannot prove that every code path is error-free. No instrumented coverage percentage or real-site accuracy is claimed.

Understand the system first

You do not need to start every component to see the website. Add services as the features need them. The downloaded reference archives are read-only material; run the application from voltarax-web.

Component	Purpose	Needed for
Next.js frontend, port 3000	Public pages and dashboard in English/Hindi	Every browser session
Express backend, port 5000	Accounts, permissions, telemetry and feature APIs	Connected dashboard
Firebase RTDB	Accounts, device registry, telemetry, relay state and AI output	Connected backend
PostgreSQL	Saved alerts, work orders, inspections, reports and orders	Persistent operational features
Redis	Shared rate limits and notification queues	Multiple API processes / queue transport
Python AI batch jobs	Current-input estimates, validation and drift	Optional AI features
NodeMCU, sensors, relay, camera/Pi	Actual measurements and physical actions	Field installation only

Browser -> Next.js /api proxy -> Express -> Firebase / PostgreSQL / Redis. NodeMCU -> Firebase directly, or the optional authenticated HTTP bridge. AI reads reviewed telemetry and can publish validated output when explicitly enabled.

Choose a running mode

A. Public-page preview: simplest first start, no hardware or Firebase credentials needed. Dashboard/API features remain unavailable. Follow page 3.

B. Automated local demonstration: isolated demo Firebase emulator, temporary SQL, real product auth/API routes and fake mail/payment/device responses. Tests open and close their own servers. Follow page 4. It is not a permanent interactive installation.

C. Connected test installation: your selected non-production Firebase project, configured credentials, real storage and deliberately enabled test integrations. Follow pages 5-7. Add AI/hardware only after this works.

Prerequisites and this computer

Node 24.19.0/npm 11.17.0 and Python 3.13.15 were used. Java 21 and the cached official RTDB emulator are required for mode B. Node's native TypeScript support is used by the backend schema. Install both service locks separately; the root package is not an all-service launcher.

This host has 4 GB RAM. Existing checks passed with one build worker and a 768 MB Node build heap; keep heavy build, AI, emulator and browser tasks sequential. This is an observation, not a production capacity estimate. Docker Compose is installed but its engine is currently unavailable. Start Docker Desktop before mode C's container path; image builds/runtime remain unverified here.

A. Open the public website

Use a PowerShell terminal. These paths match this checkout. Keep the terminal running while using the preview.

1. Install the frontend lock once

```
$project = 'C:/Users/JARVIS/Desktop/VOLTARAX'
Set-Location $project
python tools/run-check.py --cwd voltarax-web/frontend `
  --name local-frontend-install -- npm ci
if ($LASTEXITCODE -ne 0) { throw 'Frontend install failed' }
```

The helper limits installation to ten minutes and saves its output. Run it again after a reviewed lock-file change or on a new machine. Do not replace the lock with an unreviewed dependency update.

2. Preserve or create local configuration

```
Set-Location "$project/voltarax-web/frontend"
if (-not (Test-Path .env.local)) {
  Copy-Item .env.example .env.local
}
npm run dev -- --hostname localhost
```

Open http://localhost:3000 for English or http://localhost:3000/hi for Hindi. A first page can take longer while development code compiles. Stop the server with Ctrl+C in its terminal.

Public pages can be inspected without accounts. Backend-dependent forms, login and dashboard data need mode C or the owned automated fixtures in mode B. A page opening does not establish complete software operation.

Optional: production-style local preview

Stop the development server first. Build for the intended backend URL; do not reuse the automated demo build for a normal installation.

```
$env:NEXT_TELEMETRY_DISABLED = '1'
$env:NODE_OPTIONS = '--max-old-space-size=768'
python ../../tools/run-check.py --cwd . `
  --name local-production-build -- npm run build
if ($LASTEXITCODE -ne 0) { throw 'Build failed' }
$env:HOSTNAME = 'localhost'
$env:PORT = '3000'
npm start
```

The build automatically stages public/static files in .next/standalone. npm start runs that server; do not manually delete build directories. BACKEND_URL is compiled into the proxy: change it in .env.local and rebuild whenever the API destination changes. Build-time public variables are visible to users; credentials never belong in NEXT_PUBLIC_*.

B. Repeat the safe local checks

Run from the workspace root after installing both service locks. Stop a manual frontend on the fixture ports first. The helpers refuse occupied ports rather than stopping their owners. Use a fresh result label if preserving previous evidence.

1. Source checks and demo build

```
Set-Location 'C:/Users/JARVIS/Desktop/VOLTARAX'
python tools/run-check.py --cwd voltarax-web/backend `
  --name local-backend-install -- npm ci
python tools/run-check.py --cwd voltarax-web/backend `
  --name local-backend -- npm test
python tools/run-check.py --cwd voltarax-web/frontend `
  --name local-lint -- npm run lint
$env:LOCALE_SMOKE_LABEL = 'local-build'
python tools/run-check.py --cwd . --name local-build -- `
  python tools/verify-locale-routes.py --build
```

Check each exit code before continuing. This special build fixes the API target to the owned loopback fixture on port 38475. It is for these tests only, not the normal port-5000 API. Rebuild as on page 3 afterward for normal use.

2. Public and actual-API browser checks

```
$env:FRONTEND_CHECK_SERVER = 'production'
$env:FRONTEND_CHECK_BROWSER = 'true'
$env:LOCALE_SMOKE_LABEL = 'local-public'
$env:FRONTEND_CHECK_GREP = '@public|@phase11'
python tools/run-check.py --cwd . --name local-public -- `
  python tools/verify-locale-routes.py
$env:BACKEND_BROWSER_LABEL = 'local-api'
python tools/run-check.py --cwd . --name local-api -- `
  python tools/run-local-database.py `
  --log docs/test-results/local-api-java.log -- `
  python tools/verify-backend-browser.py
```

Install the isolated axe tool from the frontend directory with npm install --prefix ../../.local-tools/a11y --no-save --package-lock=false --ignore-scripts --no-audit --no-fund axe-core@4.14.0. This MPL-2.0 tool stays outside the product bundle.

Windows uses installed Edge with a separate temporary profile. On Linux, install Playwright Chromium with the frontend's locked tool. Java and the official database emulator JAR must exist; download it using the backend's Firebase CLI setup command for demo-voltarax if missing. No real project or rules deployment is needed.

Ports: frontend localhost:38473, fixture API 127.0.0.1:38475, emulator 127.0.0.1:9000. The helpers own their process trees and shut them down. The public/UI cases use synthetic API fixtures and include the 15-screen state/axe matrix. Four separate actual-API cases use product account, SQL, chat and signature routes with fake external responses. They check OTP/session/site permissions, relay state and test checkout replay, not physical devices or Razorpay-hosted Checkout. SMTP is disabled.

C. Configure the connected API

Use a separate, deliberately selected Firebase test project. This path connects to that project and is different from the automatic emulator tests. Do not reuse live credentials or point migration/bootstrap commands at an unreviewed database.

1. Prepare private service settings

```
$project = 'C:/Users/JARVIS/Desktop/VOLTARAX'
Set-Location "$project/voltarax-web/backend"
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
python ../../tools/run-check.py --cwd . `
  --name connected-backend-install -- npm ci
```

Edit the ignored backend .env privately. Replace the example Firebase project, RTDB URL and service-account fields, or provide the complete service-account JSON. Never share those values in source, screenshots or logs. Supply strong, independent JWT_SECRET and OTP_SECRET (at least 32 random characters each). No example secret or default password is usable.

For local HTTP development, set:

```
NODE_ENV=development
PORT=5000
FRONTEND_URL=http://localhost:3000
CORS_ORIGINS=http://localhost:3000
TRUST_PROXY_HOPS=0
BOOTSTRAP_ADMIN_ENABLED=false
EMAIL_ENABLED=false
NOTIFICATIONS_ENABLED=false
PAYMENTS_ENABLED=false
CAMERA_ENABLED=false
RELAY_CONTROL_ENABLED=false
```

Set the actual SOLAR_SITE_ID/SOLAR_DEVICE_ID consistently in backend, AI and firmware. Leave unknown panel count/tariff values blank; do not invent readings or deployment facts. Keep Firebase emulator variables blank for this connected path. Optional DATABASE_URL/REDIS_URL can initially stay blank; their dependent features will be unavailable.

2. Start backend, then frontend

In the backend terminal run npm run dev. Its /health endpoint should return 200 at http://127.0.0.1:5000/health. /ready must return 200 only when the configured dependencies respond. Inspect safe errors if startup fails; do not post your .env file.

The current backend listener accepts all network interfaces. Keep port 5000 private with the host firewall or an isolated development network. The frontend preview binds localhost explicitly. Production cookies are Secure: use the intended HTTPS/proxy installation for production mode, and verify actual browser cookie behavior there.

In a second terminal follow page 3. Set frontend BACKEND_URL=http://127.0.0.1:5000 in .env.local. Use localhost consistently for the browser origin. Visit /healthz for frontend liveness, /health for proxied API liveness, and /api/ready for configured dependencies. Readiness does not prove fresh sensors, mail, payments, map approval or AI accuracy.

C. Accounts and persistent features

Email verification and first administrator

Interactive registration/sign-in verification requires an actual sender. Configure SMTP_HOST, SMTP_PORT (465 or 587), SMTP_USER, SMTP_PASS and SMTP_FROM in the private backend settings. Keep EMAIL_ENABLED=false until the operator intentionally authorizes delivery to a controlled test address, then enable and restart. The automated demo uses a fake sender; it does not give the connected server a mailbox or an OTP bypass.

There is no default administrator. Only on the chosen test Firebase project, set BOOTSTRAP_ADMIN_ENABLED=true plus private ADMIN_NAME, ADMIN_EMAIL and a unique ADMIN_PASSWORD of 16-72 characters (at most 72 UTF-8 bytes). Then, once:

```
Set-Location 'C:/Users/JARVIS/Desktop/VOLTARAX/voltarax-web/backend'
npm run seed
```

Immediately disable bootstrap, remove ADMIN_PASSWORD and restart the API. Sign in and verify email. The guard refuses an existing administrator or a previously claimed bootstrap. Investigate a partial claim rather than resetting/deleting the lock. An administrator assigns verified viewers their site/role from Dashboard > Users with a target-bound OTP. Unassigned users must not receive site telemetry.

SQL storage and Redis

PostgreSQL is needed for saved alerts/work orders/inspections/reports and commerce. Redis is needed for shared limits and real notification queues. PGlite used by tests is temporary test storage, not the connected server's database.

With Docker Desktop's engine available, create the root voltarax-web/.env from its .env.example without overwriting an existing private file. Fill independent POSTGRES_PASSWORD and REDIS_PASSWORD. For native backend processes, configure the service .env URLs using 127.0.0.1:15432 (PostgreSQL) and 127.0.0.1:16379 (Redis). URL-encode password characters. The Compose backend instead uses postgres:5432 and redis:6379 internally; do not mix these two address sets.

```
Set-Location 'C:/Users/JARVIS/Desktop/VOLTARAX/voltarax-web'
docker compose --profile ai config --quiet
docker compose up -d postgres redis
```

The full Compose parser requires DATABASE_URL and REDIS_URL even when starting only storage; root values should use internal service names. Native service .env values separately use the published loopback ports. Never print the resolved Compose configuration with secrets.

Review backend/db/migrations, select the intended empty test database and take a backup before the explicit migration. In the native backend terminal, set ALLOW_RELATIONAL_MIGRATIONS=true for npm run db:migrate only; remove that terminal override afterward. Runtime does not migrate. Restart the API and verify /api/ready, then create a harmless saved work order. A configured missing schema must fail readiness, not silently create tables.

C. Run the container stack

This is an operator procedure prepared from the actual files. Compose syntax and offline runtime inputs were checked; no image was built or started in this review because the Docker engine is unavailable.

Configure the root environment

Use voltarax-web/.env for Compose interpolation. Service .env files are not automatically loaded by Compose. Supply the selected test Firebase credentials, independent JWT/OTP and storage secrets, and consistent site/device settings. DATABASE_URL must address postgres:5432; REDIS_URL must address redis:6379 with URL-encoded credentials. Keep every optional integration disabled at first. AI has separate AI_FIREBASE_* inputs.

Build and start in order

```
Set-Location 'C:/Users/JARVIS/Desktop/VOLTARAX/voltarax-web'
docker compose --profile ai config --quiet
docker compose build backend frontend
docker compose up -d postgres redis
# After reviewing migrations, backup and the selected test database:
docker compose run --rm --no-deps `
  -e ALLOW_RELATIONAL_MIGRATIONS=true backend npm run db:migrate
docker compose up -d backend frontend
docker compose ps
```

Check each command succeeds. Health and dependency startup can take time. Open http://localhost:3000 for a local connectivity check. For production-mode sessions, use the reviewed HTTPS origin/proxy and verify Secure cookies; do not weaken cookie policy to make an HTTP test pass. Keep the backend/storage private. The frontend uses http://backend:5000 internally, built into its proxy.

First-admin setup is separate: after configuring the private one-time bootstrap values and deliberately intended mail delivery, run docker compose run --rm --no-deps backend npm run seed once. Disable bootstrap/remove its password in the root environment afterward and recreate the backend so its running configuration no longer contains those values. The seed script is now included in the image, but its actual container execution still needs verification.

Health, logs and shutdown

```
docker compose ps
docker compose logs --tail 80 backend frontend
docker compose stop frontend backend redis postgres
```

Review logs privately; share only redacted errors. Stop preserves named volumes. Do not remove volumes or run destructive cleanup as troubleshooting. To undo an application rollout, use a recorded schema-compatible image/configuration and the reviewed recovery procedure, not an automatic destructive migration.

For a hosted installation use docs/RELEASE_OPERATIONS.md: its Coolify settings, TLS, private service network, environment scopes, health checks, monitoring, backups and rollback still need actual host validation and separate deployment authorization. No hosting account, domain, uptime or recovery capability is assumed.

Add AI and hardware gradually

Python AI setup and checks

From the AI directory, create an isolated environment if it does not already exist. Use Python 3.13, then install the recorded lock:

```
Set-Location 'C:/Users/JARVIS/Desktop/VOLTARAX/voltarax-web/ai-engine'
if (-not (Test-Path ../../.local-tools/ai-python/Scripts/python.exe)) {
  python -m venv ../../.local-tools/ai-python
}
$ai = (Resolve-Path ../../.local-tools/ai-python/Scripts/python.exe).Path
& $ai -m pip install --only-binary=:all: -r requirements.lock.txt
& $ai -m pytest tests -q -p no:cacheprovider
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
```

Bound slow installs/tests with tools/run-check.py as on earlier pages. Set actual site/device, separate least-privilege Firebase credentials and private artifact permissions in AI .env. Remote reads and publication default off; enabling them is an intentional operator action only after real-data review. With reviewed settings, run & $ai scripts/run_once.py for eligible training, then & $ai scripts/predict_once.py for inference. Insufficient/stale/dark input or absent models must remain unavailable. The website does not start these jobs automatically.

The optional assistant provider is separate: CHAT_PROVIDER defaults disabled. Reviewed EN/HI FAQ replies still work. A deliberately installed local Ollama model can select reviewed source IDs only, using the loopback origin/model fields in the backend example. User text/personal context is never sent to it. Actual Ollama execution was not performed; see docs/CHAT_ASSISTANT.md.

The container equivalent is the optional ai profile; build it separately, then use docker compose --profile ai run --rm ai for its default inference batch. It exits rather than becoming an HTTP server. The corrected package passes offline imports, not an image/runtime test. Operator scheduling is in ai-engine/scripts/schedule_cron.md: non-overlapping minute inference and hourly drift/training, each capped at ten minutes. No schedule was activated.

AI estimates current conditions. Future weather-based forecasting, calibrated field accuracy and automatic confirmed fault diagnosis remain unfinished/unverified. Do not treat synthetic test metrics as site performance.

Hardware is a separate commissioning task

Follow hardware/nodemcu/README.md and docs/MANUAL_STEPS.md for sensors, pin assignments, private secrets and actual Firebase device UID/rules. Compile_check uses placeholder credentials and must never be uploaded. The native helper tests do not exercise Wi-Fi or electrical components. Flash only the reviewed device configuration after safe wiring/calibration; no flash occurred here.

Verify actual telemetry timestamps/ranges and freshness first. Register/rotate keys through the guarded administrator workflow for bridge devices. Enable the relay only after polarity, startup/disconnect behavior and physical contact checks. A GPIO acknowledgement does not prove contact state. Camera/coating records require configured storage and actual approved inputs.

Map geometry, real SMTP, Redis queue behavior, Razorpay TEST provider operation and backups likewise need separate checks. Do not enable everything at once: verify one dependency/feature and its failure behavior before adding the next.

Troubleshooting and remaining work

Symptom	What to check
Website opens, dashboard does not	API running, exact frontend origin, proxy target, verified account and site assignment
API startup fails	Valid private JWT secret, Firebase test-project credentials/RTDB URL, configured Redis availability; inspect safe local log
Login/OTP unavailable	SMTP intentionally enabled/configured, controlled test recipient, separate OTP secret; demo fake mail is not connected mail
Login works but session disappears	Consistent localhost origin, NODE_ENV and Secure cookie/HTTPS behavior; no frontend localStorage token is expected
API proxy fails after normal start	Demo build targets port 38475; rebuild with intended BACKEND_URL, normally 127.0.0.1:5000 for native development
/health is 200, /api/ready is 503	Configured Firebase/SQL/Redis connection or SQL schema; liveness is not dependency readiness
Saved features unavailable	DATABASE_URL, reviewed migrations, SQL permissions; optional storage is not created automatically
Docker pipe/daemon error	Start the installed Docker Desktop engine; config parsing alone does not start it
Emulator/browser fails to start	Java/official JAR/Edge or Chromium installed, fixture ports free; do not terminate unknown owners
AI unavailable	Explicit read/publish settings, identity, fresh sufficient readings and trusted validated models; not a website startup failure
No map outline, prices or camera	Approved boundary/real business prices/private camera inputs missing; defaults deliberately remain unavailable/off
Build or tests exceed ten minutes	Inspect saved log and resource pressure, run heavy jobs sequentially; retain failed evidence rather than hiding it

What remains before a real installation

The production audit still contains eight moderate backend SDK-tree package matches. The application uses RTDB, but that narrower use does not prove those installed dependencies are harmless. Exact advisory exposure and upgrade candidates are in docs/KNOWN_ISSUES.md; do not force npm's incompatible downgrade suggestions. This review did not change dependency locks.

Real Firebase identity/rules and migration, PostgreSQL/Redis server/concurrency, SMTP delivery, hardware/camera/relay, approved web-sized India boundaries, field AI data, Razorpay-hosted TEST Checkout, fluent Hindi/assistive-technology review, business/legal/tax approval, actual containers/hosted CI, TLS, monitoring and isolated backup restoration remain unverified or require owner input. Full Hindi PDF/email output and weather-ahead forecasting remain unfinished.

Recommended order: public preview -> automated local checks -> isolated Firebase/API/accounts -> SQL/Redis -> one real feature at a time -> hardware/AI -> reviewed hosted test installation and recovery drill. Preserve existing data throughout. No real mail/payment, deployment, user-data migration/deletion or hardware flashing was performed in this review.

Current evidence: docs/test-results/phase11-gate-summary.json. Baseline second-review evidence: reverify-summary.json. Current tasks: docs/PROGRESS.md. External steps: docs/MANUAL_STEPS.md. Detailed operations: docs/RELEASE_OPERATIONS.md. The updated completion source and PHASE11_GAP_MAP.md distinguish Phase 11 local work from unchanged external requirements. The original phase tags remain historical checkpoints.